

Chapter 10: Production Debugging, Profiling & Memory Leaks Lab

The Difference Between Junior and Principal Engineers Anyone can write code that works under ideal conditions. A senior engineer shines when code is burning in production: CPU usage is pegged at 100%, memory consumption creeps steadily toward an Out-Of-Memory (OOM) kernel kill, or worker threads mysteriously freeze in an undetectable deadlock.

This chapter is a hands-on diagnostic laboratory covering production debugging, CPU profiling, and memory leak triage in CPython.

1. Surgical Debugging with `pdb` and `breakpoint()`

Modern Python (3.7+) introduced `breakpoint()`, which calls the sys-hook `sys.breakpointhook()`. By default, this drops into the `pdb` (Python Debugger) REPL.

Production Post-Mortem Debugging: Catching Crashes After Death

In production, you often cannot pause execution interactively. Instead, your application crashes with an unhandled exception. **Post-mortem debugging** inspects the exact stack frame and local variables at the moment of death.

Essential `pdb` Command Cheat-Sheet

When dropped into the debugger:

- `w` (where): Prints the complete call stack trace from top to bottom.
- `u` (up) / `d` (down): Navigates up or down the call stack frames to inspect caller variables.
- `p <expr>` / `pp <expr>`: Evaluates and pretty-prints any expression in the current scope.
- `l .` (list): Displays the source code centered around the current execution line.
- `c` (continue): Resumes execution until the next breakpoint.
- `q` (quit): Immediately terminates the Python process.

TIP

Run any failing script with `python -m pdb -c continue script.py`. Python will run at native speed without stopping until an unhandled exception is raised, at which point it automatically halts and enters post-mortem mode right at the offending line!

2. Profiling CPU Hotspots with `cProfile` and `pstats`

Never optimize without measurement. "Intuition" about where an algorithm spends its time is almost always wrong.

Profiling a Slow Pipeline

Consider this data-processing function with an intentional bottleneck:

Deciphering `pstats` Output

- `ncalls`: Number of invocations.
- `tottime`: Total time spent strictly inside this function (excluding calls to other functions).
- `cumtime`: Cumulative time spent in this function **including** all sub-functions it called.
- Notice: the profiler attributes time to `slow_string_concatenation` and `fast_string_join` separately, which is exactly how you locate a hot spot. The timings above are illustrative output from one run. **Do not expect a large gap on CPython:** CPython's `str +=` has an in-place fast path when the string has a single reference, so `labs/05_benchmarks.py` measured only about 1.2x for `+=` versus `join` on Python 3.11. `join` is still the portable idiom because the fast path is an implementation detail: it disappears for `self.s = self.s + x`, for other interpreters, and whenever a second reference exists, and then the loop really is quadratic.

3. Hunting Production Memory Leaks with `tracemalloc`

Because CPython uses Reference Counting supplemented by a Generational Cyclic Garbage Collector, many developers assume memory leaks are impossible. This is a dangerous myth.

How Memory Leaks Happen in Python:

1. **Global Collections:** Appending records to module-level lists or dicts without an eviction policy (unbounded cache).
2. **Circular References with `__del__` (Pre-Python 3.4) or lingering closures.**
3. **C-Extension Memory Leaks:** C code (via Cython, ctypes, or C++ bindings) allocating memory with `malloc()` without freeing it.

Step-by-Step Leak Detection Lab

[Redacted]

Output:

[Redacted]

Line 19 (`self.payload = bytearray( ... )`) is instantly pinpointed as the source of the 50 MB leak!

The Fix: Weak References (`weakref`)

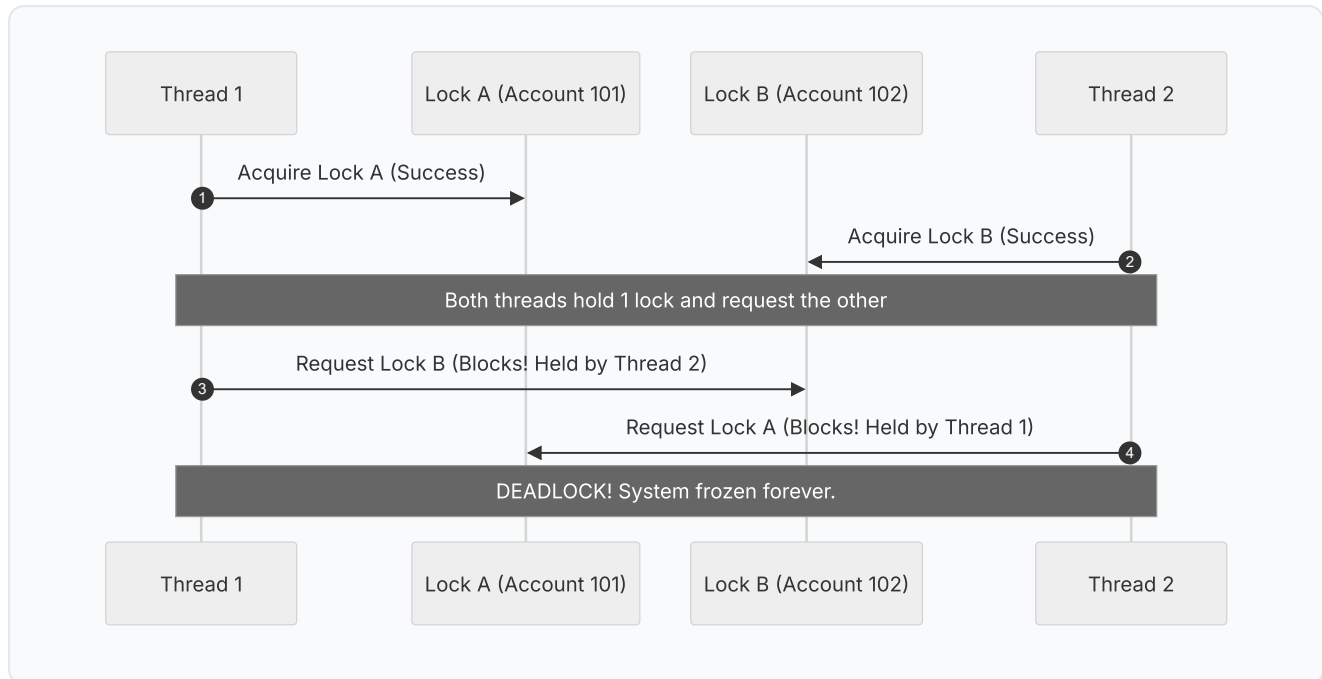
When an event broker or cache must reference an object without preventing its garbage collection:

[Redacted]

4. Diagnosing Deadlocks with `faulthandler`

When your multi-threaded application suddenly hangs at 0% CPU and stops accepting requests, you are almost certainly facing a **Deadlock**.

What Causes a Deadlock? (AB-BA Lock Inversion)



Reproducing and Inspecting with `faulthandler`

`faulthandler` is a standard library module that talks directly to the OS kernel signal handlers to dump the execution stack of **every active thread**, even when Python's GIL is frozen.

Traceback Dump in Terminal:

You can immediately identify:

- `Worker-1` is blocked waiting on `line 12` (`with lock_b:`).
- `Worker-2` is blocked waiting on `line 18` (`with lock_a:`).

5. Senior Takeaways & Lab Exercise

- Always use context managers** (`with lock:`) for acquiring locks, but establish a strict **global lock acquisition order** (e.g., sort locks by ID before acquiring) to mathematically eliminate deadlocks.
- Never use unbound callbacks or caches** without `weakref` or TTL/LRU eviction policies.

3. Keep `faulthandler` registered in production servers
(`faulthandler.register(signal.SIGUSR1)`) so you can trigger live thread stack dumps on running Kubernetes pods without restarting them!

Exercises

Three graded exercises for this chapter (two coding, one debugging) with hidden tests:



Attempt first; the reference solutions are in `exercises/_answers/ch10.py`.

Further Reading

- [tracemalloc](#)
- [cProfile and profile](#)
- [py-spy](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- ▶ 1. Which tool shows where Python allocated memory over time?
- ▶ 2. How do you find a CPU hotspot?
- ▶ 3. A long-running service's memory only grows. First three suspects?
- ▶ 4. Why measure before optimising?